



Máy học: Tăng cường (Boosting)

Chương 7 (Phần 3)

Bài giảng Giới thiệu Bậc Đại học

Mã nguồn: Tính toán Feature Importances

```
from sklearn.datasets import load_iris
iris = load_iris(as_frame=True)
rnd_clf = RandomForestClassifier(n_estimators=500, random_state=42)
rnd_clf.fit(iris.data, iris.target)

for score, name in
zip(rnd_clf.feature_importances_,
iris.data.columns):
    print(round(score, 2), name)
```



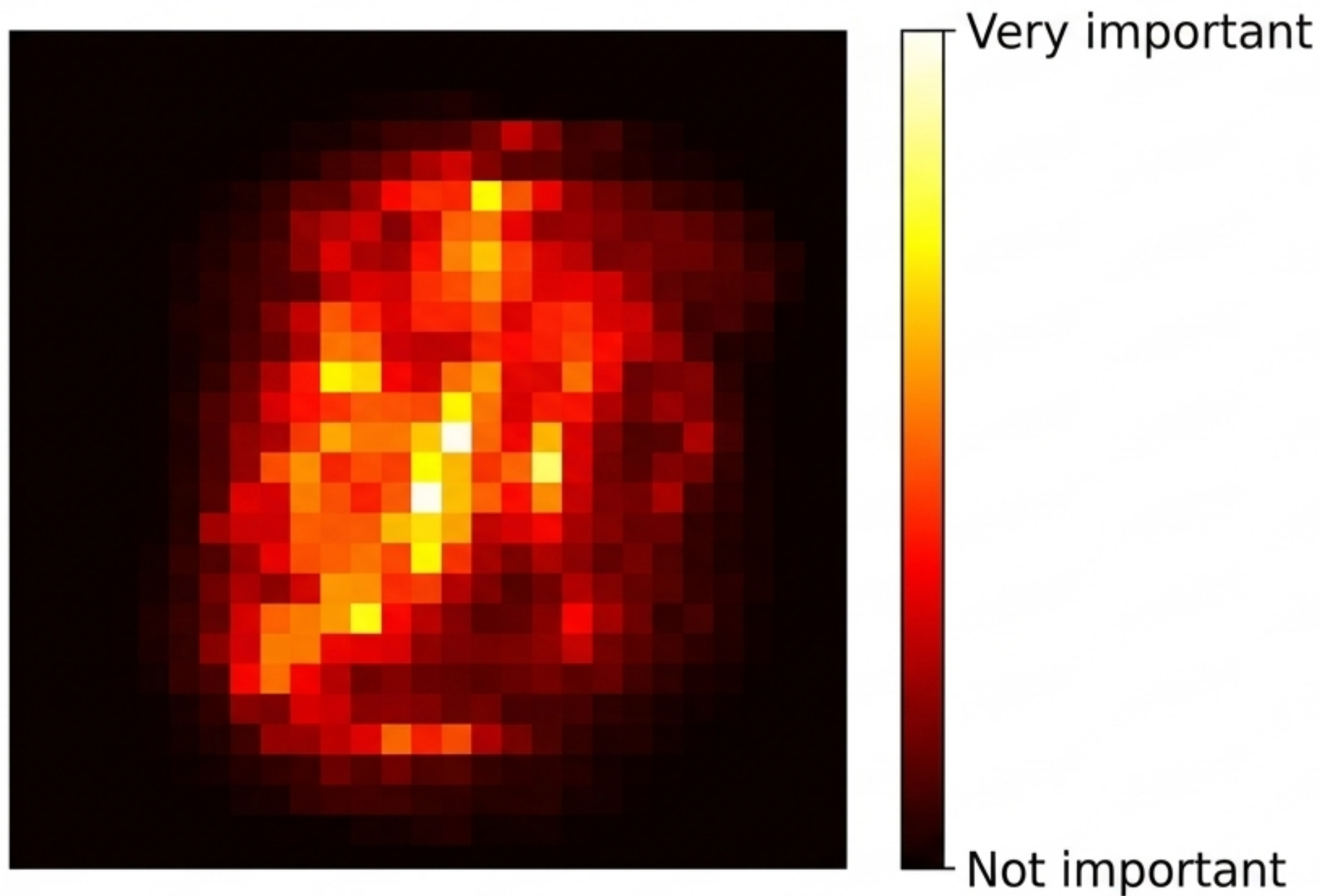
0.11 sepal length (cm)

0.02 sepal width (cm)

0.44 petal length (cm) ← <-- Quan trọng nhất

0.42 petal width (cm) ← <-- Quan trọng thứ hai

Tầm quan trọng pixel MNIST



Hình 7-6. Tầm quan trọng của pixel trên ảnh MNIST

Khái niệm Tăng cường (Boosting)



Boosting (Tăng cường giả thuyết) biến các bộ học yếu thành bộ học mạnh.

Huấn luyện các bộ dự đoán theo trình tự, mỗi bộ sau sẽ cố gắng sửa chữa lỗi của bộ tiền nhiệm.

1 AdaBoost (Adaptive Boosting)

2 Gradient Boosting

Trực giác về AdaBoost (Adaptive Boosting)



Tập trung nhiều hơn vào các trường hợp huấn luyện bị phân loại sai bởi bộ tiền nhiệm.



Huấn luyện mô hình cơ sở (thường là Cây quyết định một nút) và dự đoán.



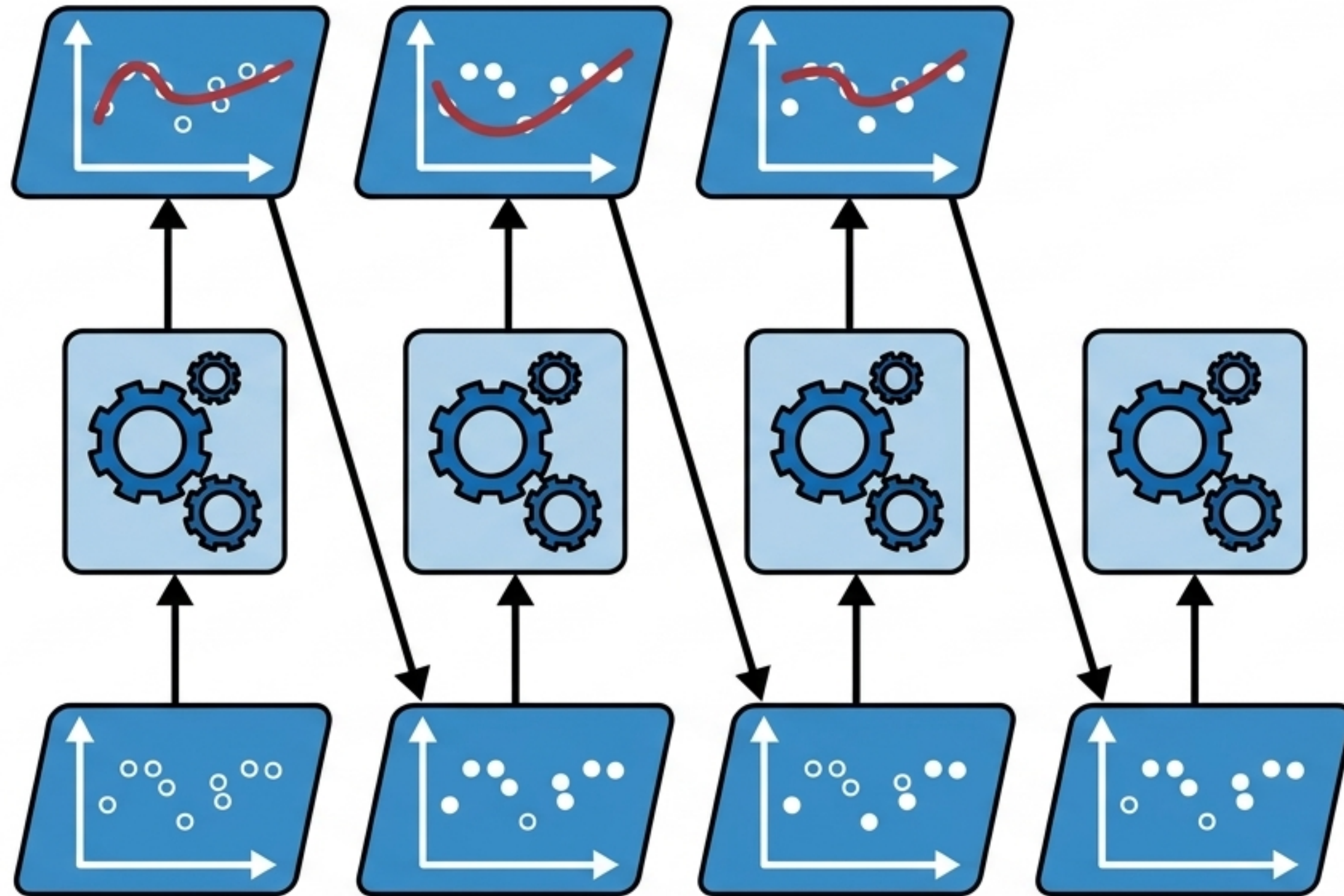
Tăng trọng số tương đối của các trường hợp bị sai.



Huấn luyện mô hình thứ 2 dựa trên trọng số mới cập nhật, dự đoán, tăng trọng số, v.v.

Kỹ thuật này khá giống **Gradient Descent**, nhưng thay vì chỉnh tham số cho một mô hình, nó thêm **mô hình mới** vào tập hợp.

Cập nhật trọng số trong AdaBoost



Hình 7-7. Huấn luyện AdaBoost tuần tự với cập nhật trọng số

Phân tích trọng số trong SVM (Dữ liệu moons)



Learning Rate Lớn

Bộ dự đoán thứ 2 sửa rất mạnh các lỗi của bộ đầu tiên.



Learning Rate bằng Một nửa

Các SVM sửa đổi một cách cẩn thận và dần dần hơn.

Toán học AdaBoost: Tỷ lệ lỗi & Trọng số

$$r_j = \frac{\sum_{i=1, \hat{y}_j^{(i)} \neq y^{(i)} }^m w^{(i)}}{\sum_{i=1}^m w^{(i)}}$$

Trọng số ban đầu: $w^{(i)} = 1/m$.

Tỷ lệ lỗi có trọng số của bộ thứ j (r_j).

$$\alpha_j = \eta \log \frac{1 - r_j}{r_j}$$

Trọng số của bộ dự đoán thứ j (α_j).

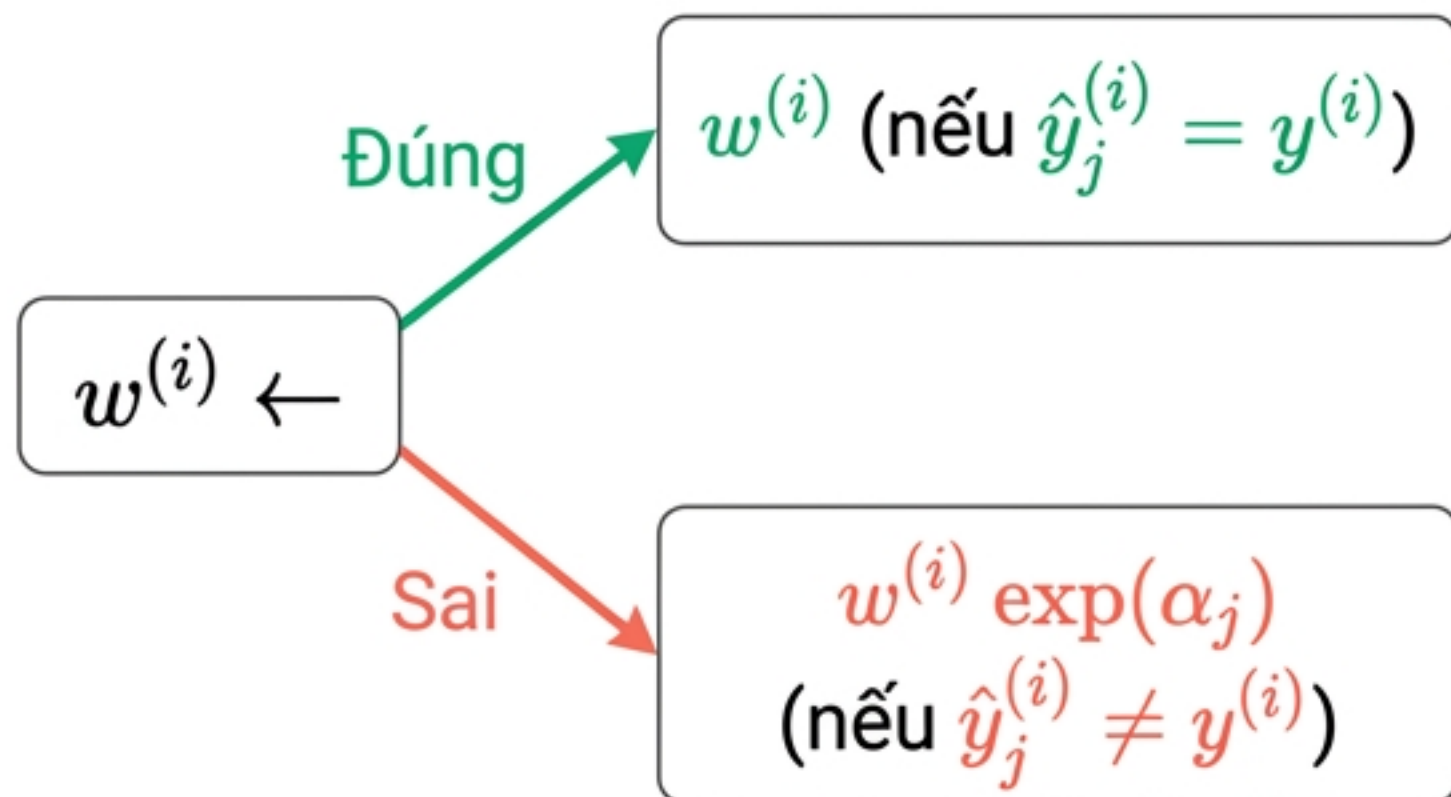


Bộ càng chính xác $\rightarrow \alpha_j$ càng cao.
Đoán bừa $\rightarrow \alpha_j \approx 0$.

η là learning rate (mặc định 1).

Toán học AdaBoost: Cập nhật & Dự đoán

Quy tắc cập nhật trọng số trường hợp ($w^{(i)}$)



*Sau đó, tất cả trọng số được chuẩn hóa (chia cho $\sum w^{(i)}$).



Dự đoán cuối cùng

$$\hat{y}(x) = \operatorname{argmax}_k \sum_{j=1, \hat{y}_j(x)=k}^N \alpha_j$$

Mã nguồn: AdaBoostClassifier



Scikit-Learn sử dụng biến thể đa lớp tên là SAMME (hoặc SAMME.R để tận dụng xác suất).

```
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier

ada_clf = AdaBoostClassifier(
    DecisionTreeClassifier(max_depth=1), \
    n_estimators=30,
    learning_rate=0.5,
    random_state=42
)
ada_clf.fit(X_train, y_train)
```

Cây 1 nút
(decision stump)

Gradient Boosting: Khác biệt cốt lõi

Tương tự AdaBoost: huấn luyện bộ sau sửa lỗi của bộ trước.



AdaBoost Nhắc lại

Thay đổi trọng số trường hợp bị sai.



Gradient Boosting

Khớp bộ dự đoán mới với các **lỗi còn lại (residual errors)** do dự đoán trước tạo ra.

Gradient Tree Boosting (GBRT): Sử dụng Cây quyết định làm bộ cơ sở.

Gradient Tree Boosting Thủ công (Phần 1)

Tạo dữ liệu và huấn luyện cây đầu tiên



```
from sklearn.tree import DecisionTreeRegressor
import numpy as np

np.random.seed(42)
X = np.random.rand(100, 1) - 0.5
y = 3 * X[:, 0] ** 2 + 0.05 * np.random.randn(100)

tree_reg1 = DecisionTreeRegressor(max_depth=2, random_state=42)
tree_reg1.fit(X, y)
```

Gradient Tree Boosting Thủ công (Phần 2)

Huấn luyện trên phần dư (residuals) và kết hợp

Phân dư 1

```
y2 = y - tree_reg1.predict(X)
```

```
tree_reg2 = DecisionTreeRegressor(max_depth=2, random_state=43)
```

```
tree_reg2.fit(X, y2)
```



Phân dư 2

```
y3 = y2 - tree_reg2.predict(X)
```

```
tree_reg3 = DecisionTreeRegressor(max_depth=2, random_state=44)
```

```
tree_reg3.fit(X, y3)
```

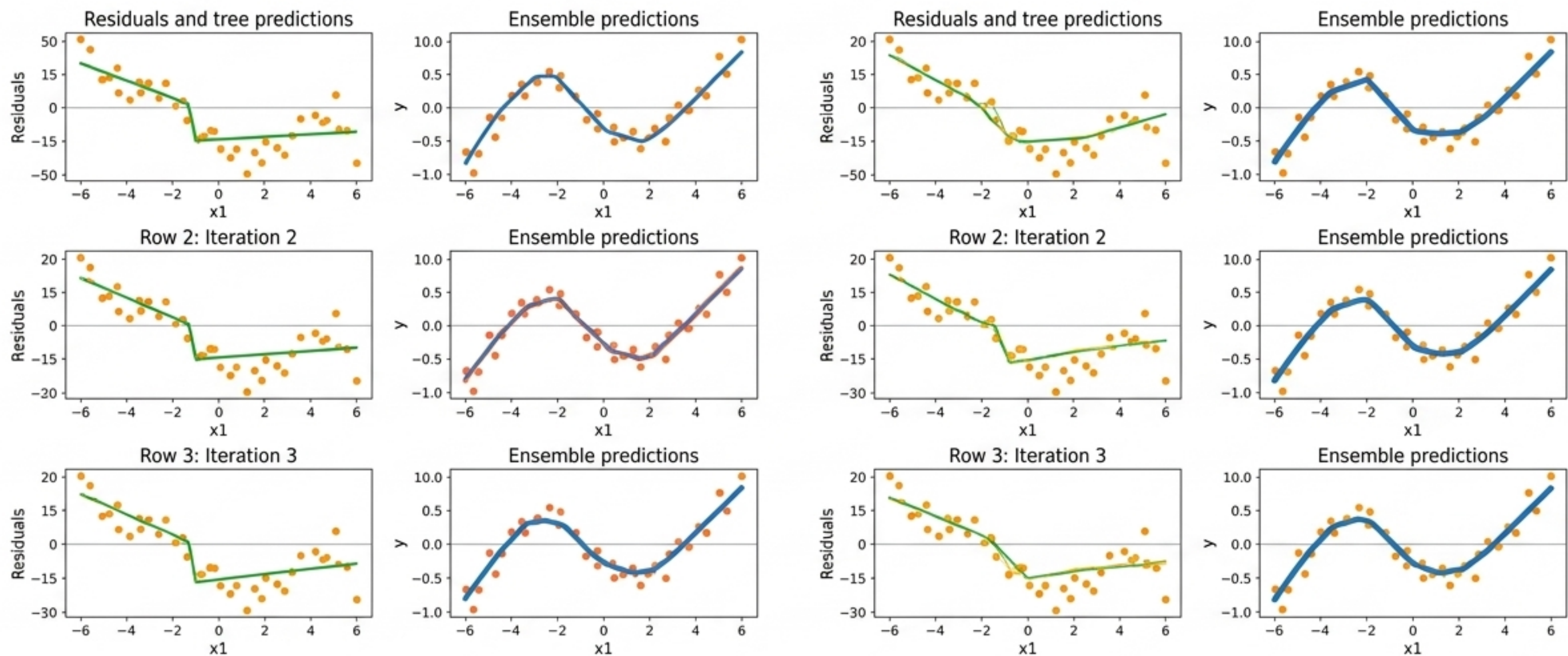
Dự đoán mới bằng tổng các cây:

```
X_new = np.array([[-0.4], [0.], [0.5]])
```

```
y_pred = sum(tree.predict(X_new) for tree in (tree_reg1, tree_reg2, tree_reg3))
```



Trực quan hóa: Quá trình huấn luyện GBRT



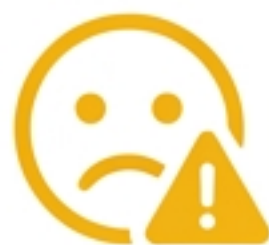
Quá trình huấn luyện GBRT

GBRT trong Scikit-Learn & Vấn đề Shrinkage



```
from sklearn.ensemble import GradientBoostingRegressor
gbrt = GradientBoostingRegressor(max_depth=2, n_estimators=3,
                                learning_rate=1.0, random_state=42)
gbrt.fit(X, y)
```

learning_rate: Kiểm soát mức độ đóng góp của từng cây.
Giá trị thấp -> Cần nhiều cây hơn (Shrinkage - Phương pháp chính quy hóa).



Số lượng cây n_estimators quá ít -> Dưới khớp (Underfitting).



Số lượng cây n_estimators quá Nhiều -> Quá khớp (Overfitting).